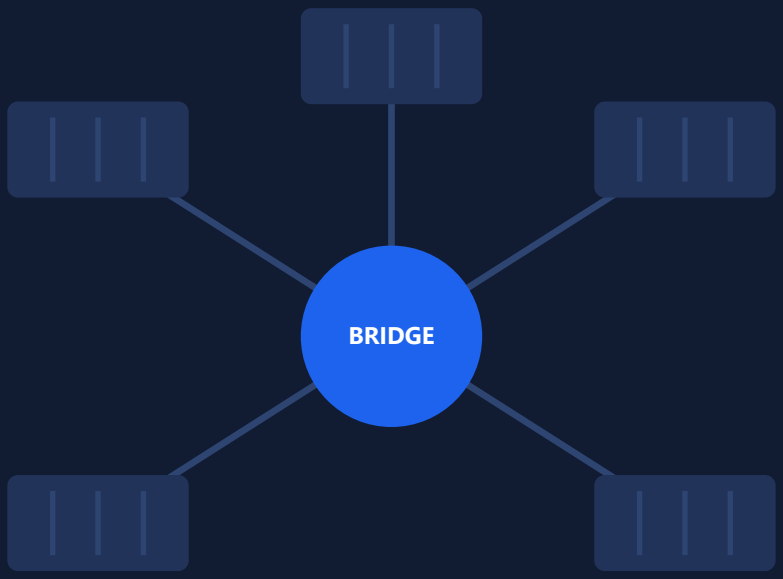
TRISHA ACADEMY

DevOps Career Track · Student Notes



Docker Networking & Container Communication

Connect containers to each other, to the host and to the world

Based on docs.docker.com/engine/network · Lab: Spring Boot Student API

Bridge networks **User-defined bridge** **Published ports** **Embedded DNS** **host & none**
container: mode **macvlan / ipvlan** **Subnets & static IPs** **host.docker.internal**
Hands-on lab **20 interview Q&A**

Contents

How to use these notes . . . 4

Part 1: How container networking works . . . 5

1.1 What is container networking? . . . 5
1.2 The three directions of traffic . . . 5
1.3 Look at it yourself . . . 6

Part 2: Network drivers . . . 7

2.1 The drivers . . . 7
2.2 Which one should I use? . . . 7

Part 3: Default bridge vs user-defined bridge . . . 8

3.1 The difference in one picture . . . 8
3.2 Try it: the default bridge . . . 8
3.3 Try it: a user-defined bridge . . . 8
3.4 Why user-defined bridges are better . . . 9

Part 4: Publishing ports . . . 10

4.1 What -p does . . . 10
4.2 All the forms of -p . . . 10
4.3 Security . . . 11

Part 5: DNS and service discovery . . . 12

5.1 The embedded DNS server . . . 12
5.2 Names a container can be found by . . . 12
5.3 DNS options for docker run . . . 12
5.4 Reaching the host from a container . . . 13

Part 6: Managing networks . . . 14

6.1 The docker network commands . . . 14
6.2 Subnet, gateway, static IP and internal networks . . . 14
6.3 One container, several networks . . . 14
6.4 Clean up . . . 15

Part 7: host, none and container: networking . . . 16

7.1 host . . . 16
7.2 none . . . 16
7.3 container: . . . 16

Part 8: macvlan and ipvlan . . . 18

8.1 macvlan . . . 18

8.2 ipvlan . . . 18

Part 9: Hands-on lab: network the devops-java-demo app . . . 19

9.1 Build the image and create the network . . . 19

9.2 Run the app on trisha-net and publish port 9090 . . . 19

9.3 Call it from three places . . . 20

9.4 Keep the app private . . . 20

9.5 Results . . . 20

9.6 Variations to try . . . 20

9.7 Clean up . . . 21

Part 10: Best practices, troubleshooting and cheat sheet . . . 22

10.1 Best practices . . . 22

10.2 Troubleshooting . . . 22

10.3 Cheat sheet . . . 23

Part 11: Top 20 interview questions . . . 24

How to use these notes

These notes teach Docker networking from zero, following the official Docker documentation (**docs.docker.com/engine/network**), and finish with a hands-on lab on the TRISHA ACADEMY **devops-java-demo** project - a Spring Boot Student API that listens on port **9090**.

- Read each part, then type every command yourself - do not copy-paste without reading.
- Commands are shown for **PowerShell** (Windows). Linux/macOS differences are noted where needed.
- Blue boxes quote the official documentation; green boxes are things to try; red boxes are common mistakes.
- Every command in Parts 3 to 9 was run on Docker Desktop (Engine 29) while writing these notes.
- You need two small images: `docker pull alpine` and `docker pull nginx:alpine`.

Part	Topic
1	How container networking works
2	Network drivers
3	Default bridge vs user-defined bridge
4	Publishing ports
5	DNS and service discovery
6	Managing networks: create, connect, inspect, remove
7	host, none and container: networking
8	macvlan and ipvlan
9	Hands-on lab: networking the devops-java-demo app
10	Best practices, troubleshooting and cheat sheet
11	Top 20 interview questions

DOCKER NETWORKING · PART 01

How container networking works

Understand what a container sees, and how it reaches other containers, the host and the internet.

1.1 What is container networking?

From the official Docker documentation

- Container networking refers to the ability for containers to connect to and communicate with each other, and with non-Docker network services.
- Containers have networking enabled by default, and they can make outgoing connections. A container has no information about what kind of network it's attached to, or whether its peers are also Docker workloads or not. A container only sees a network interface with an IP address, a gateway, a routing table, DNS services, and other networking details.

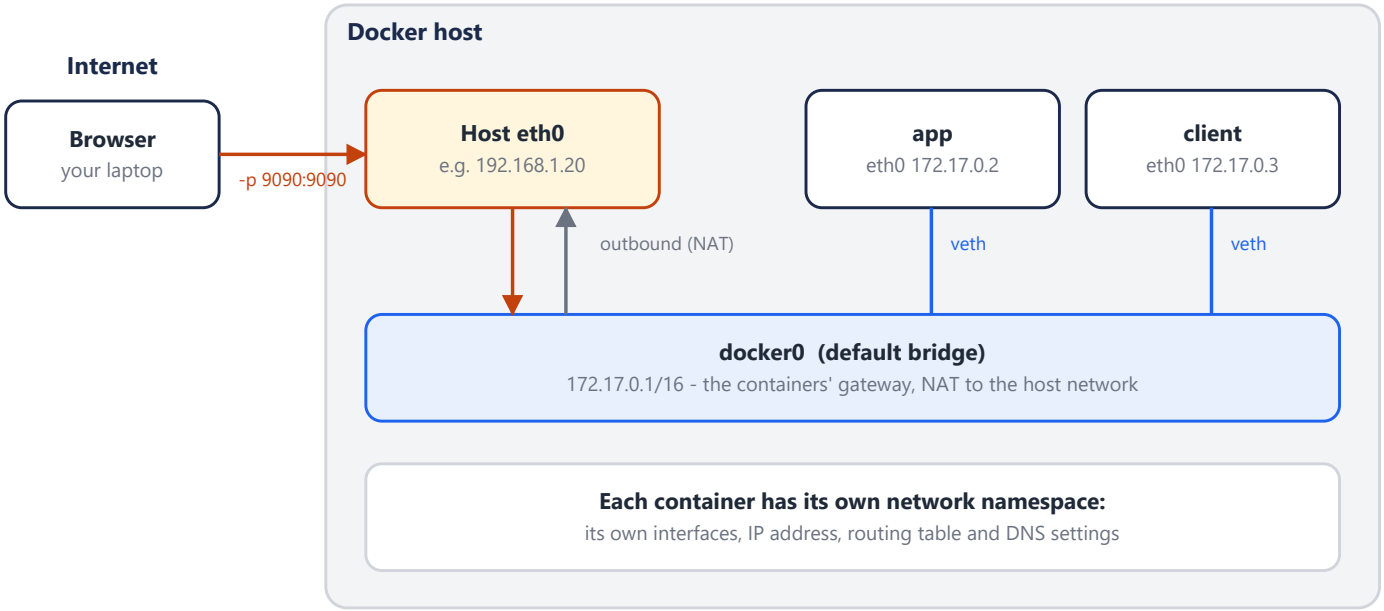


Figure 1.1 - Containers on the default bridge: private IPs, a shared gateway, NAT for outbound traffic and -p for inbound traffic

1.2 The three directions of traffic

Direction	How it works	What you do
Container -> internet	NAT through the host (masquerading)	Nothing - works by default
Container -> container	Over a shared Docker network	Put both on the same **user-defined** network and use the container name
Outside -> container	Through a published host port	Run with `-p HOST_PORT:CONTAINER_PORT`

Everyday example

A Docker network is like an office floor with its own phone extension list. People on the same floor (network) call each other by name. To reach someone from outside the building, the reception desk (published port) forwards the call to the right extension.

1.3 Look at it yourself

Three networks exist on every Docker host

```
docker network ls

NETWORK ID     NAME      DRIVER    SCOPE
b9f48163cd36   bridge    bridge    local
13ada6da870f   host      host      local
dff0714beab7   none      null      local
```

Subnet and gateway of the default bridge

```
docker network inspect bridge -f "{{range .IPAM.Config}}{{.Subnet}} gw {{.Gateway}}{{end}}"
172.17.0.0/16 gw 172.17.0.1
```

DOCKER NETWORKING · PART 02

Network drivers

Know which driver to pick for each situation.

2.1 The drivers

From the official Docker documentation
Docker's networking subsystem is pluggable, using drivers. Several drivers exist by default, and provide core networking functionality.

Driver	What it does	Use it for
`bridge`	The default. A private software bridge on the host; containers get private IPs.	Apps on one host - almost everything you do on a laptop
`host`	Removes isolation: the container uses the host's network directly.	Maximum network performance, or many ports
`none`	Completely isolates the container from the host and other containers (only loopback).	Jobs that must have no network
`macvlan`	Gives the container its own MAC address so it looks like a physical device on the LAN.	Legacy apps that expect to be directly on the physical network
`ipvlan`	Full control over IPv4/IPv6 addressing; containers share the host's MAC address.	Large LANs, or networks that limit MAC addresses
Plugins	Third-party network drivers.	Special integrations

2.2 Which one should I use?

- **User-defined bridge** - when containers on the same host must talk to each other. This is the right answer most of the time.
- **host** - when the network stack should not be isolated from the host, but the rest of the container should be.
- **none** - when the container needs no network at all.
- **macvlan / ipvlan** - when containers must appear directly on the physical network (Linux hosts).

Common mistake
Leaving everything on the **default** `bridge` network. It works for one container, but containers on it cannot find each other by name. Create your own network (Part 3).

DOCKER NETWORKING · PART 03

Default bridge vs user-defined bridge

See why your own bridge network is better than the default one.

3.1 The difference in one picture

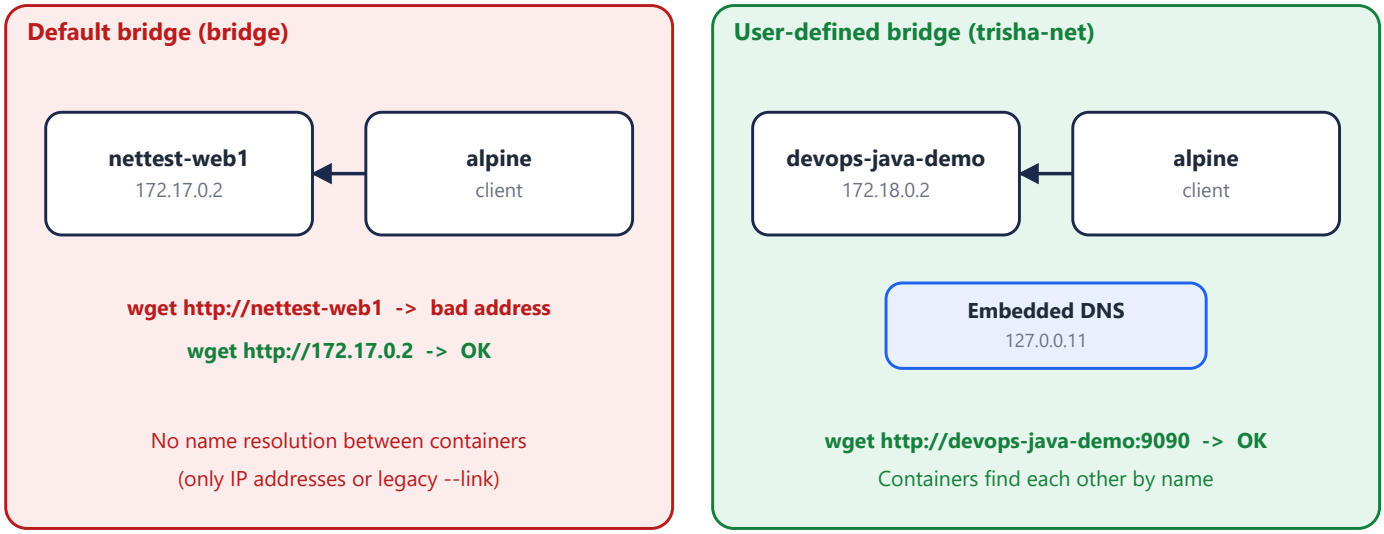


Figure 3.1 - Tested on Docker Desktop: names work only on a user-defined bridge

3.2 Try it: the default bridge

PowerShell

```
docker run -d --name nettest-web1 nginx:alpine
docker run --rm alpine wget -qO- -T 3 http://nettest-web1
wget: bad address 'nettest-web1'

# Find the IP address (Docker 29: read it per network)
docker inspect -f "{{.NetworkSettings.Networks.bridge.IPAddress}}" nettest-web1
172.17.0.2
docker run --rm alpine wget -qO- -T 3 http://172.17.0.2        # works
```

3.3 Try it: a user-defined bridge

PowerShell

```
docker network create nettest-net
docker run -d --name nettest-web2 --network nettest-net nginx:alpine
docker run --rm --network nettest-net alpine wget -qO- http://nettest-web2      # works by name
docker run --rm --network nettest-net alpine cat /etc/resolv.conf
nameserver 127.0.0.11
```

3.4 Why user-defined bridges are better

	Default bridge	User-defined bridge
Find containers by name	No (IP only, or legacy `--link`)	**Yes** - automatic DNS
Isolation	All containers started without `--network` share it	Only containers you attach
Attach / detach while running	No - recreate the container	Yes - `docker network connect`
Configuration	Global, changing it needs a Docker restart	Per network: subnet, gateway, options
Created by	Docker, automatically	You: `docker network create`

Did you know?
`--link` is a legacy feature of the default bridge. Do not use it in new setups - a user-defined network does the same job and more.

Try it yourself
Start two `nginx:alpine` containers on your own network and `wget` one from the other by name. Then repeat on the default bridge and read the error.

DOCKER NETWORKING · PART 04

Publishing ports

Make a container reachable from your browser - and only where you want it.

4.1 What -p does

From the official Docker documentation

By default, when you create or run a container using `docker create` or `docker run`, containers on bridge networks don't expose any ports to the outside world. Use the `--publish` or `-p` flag to make a port available to services outside the bridge network.

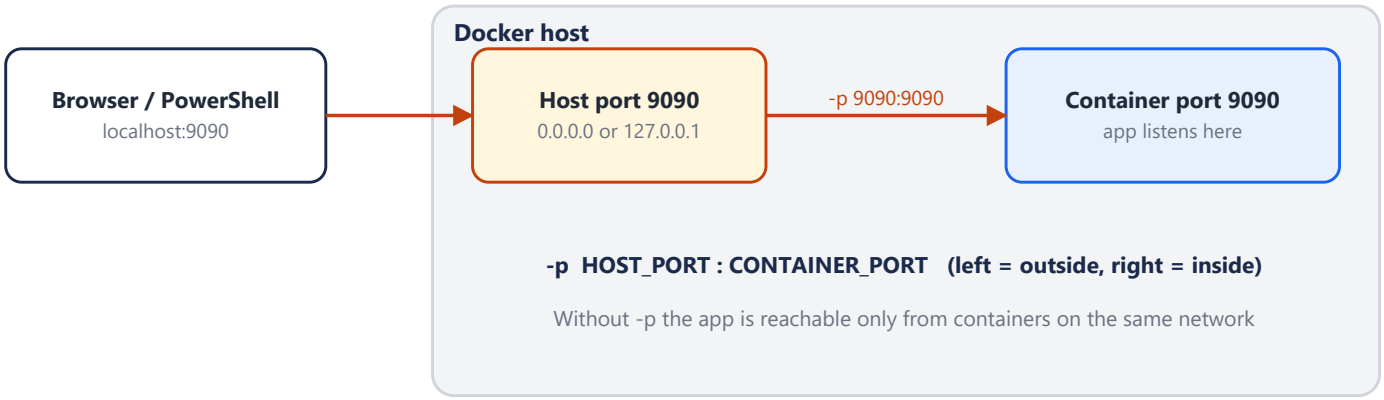


Figure 4.1 - A published port forwards host traffic into the container

4.2 All the forms of -p

Flag	Meaning
`-p 8080:80`	Host port 8080 (all interfaces) -> container TCP port 80
`-p 192.168.1.100:8080:80`	Only on the host IP 192.168.1.100
`-p 127.0.0.1:8080:80`	Only on localhost - other machines cannot connect
`-p 8080:80/udp`	UDP instead of TCP
`-p 8080:80/tcp -p 8080:80/udp`	Both TCP and UDP
`-p 80`	Container port 80 -> a **random** free host port
`-P`	Publish every `EXPOSE`d port to random host ports

Check which ports are published

```
docker run -d --name nettest-web3 -p 127.0.0.1:8089:80 nginx:alpine
docker port nettest-web3
80/tcp -> 127.0.0.1:8089

docker run -d --name nettest-web4 -P nginx:alpine
docker port nettest-web4
80/tcp -> 0.0.0.0:32768
```

4.3 Security

Official warning
Publishing container ports is insecure by default. Meaning, when you publish a container's ports it becomes available not only to the Docker host, but to the outside world as well. If you include the localhost IP address (127.0.0.1, or ::1) with the publish flag, only the Docker host and its containers can access the published container port.

Common mistake
Thinking `EXPOSE 9090` in the Dockerfile publishes the port. `EXPOSE` is only documentation (and the list used by `-P`). Nothing is reachable from the host until you use `-p` or `-P`.

Firewall surprise
On Linux servers, Docker writes its own firewall (iptables) rules. A published port can be reachable from the internet even when `ufw` seems to block it. Publish to `127.0.0.1` or use a cloud security group.

Did you know?
Containers on the same network never need `-p` to talk to each other - they use the container port directly, e.g. `http://devops-java-demo:9090`.

DOCKER NETWORKING · PART 05

DNS and service discovery

Find containers by name and control how names are resolved.

5.1 The embedded DNS server

From the official Docker documentation

Containers that use the default bridge network get a copy of the host's /etc/resolv.conf. Containers on custom networks use Docker's embedded DNS server, which forwards external DNS lookups to the DNS servers configured on the host.

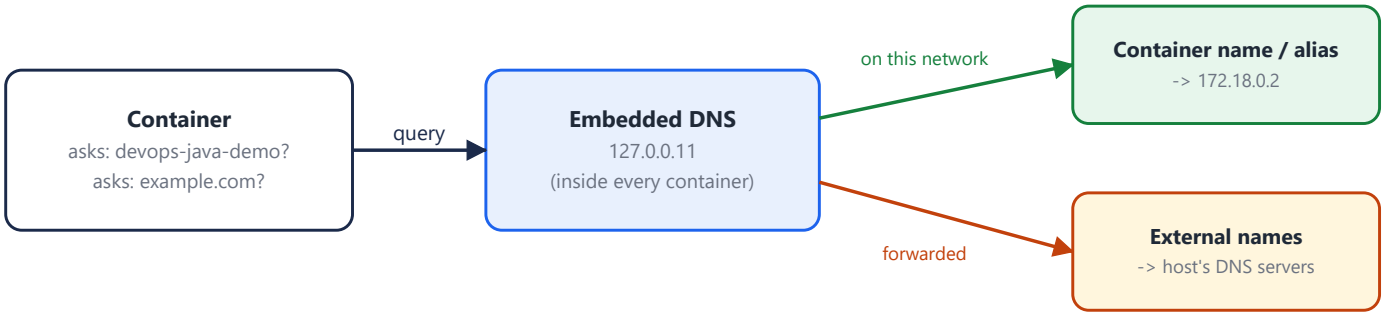


Figure 5.1 - 127.0.0.11 answers container names itself and forwards everything else

5.2 Names a container can be found by

Name	How	Example
Container name	`--name`	`http://devops-java-demo:9090`
Network alias	`--network-alias` (several containers may share one)	`http://webalias`
Container ID	Short ID also resolves	`http://eb6861818895`

Network alias

```
docker run -d --name nettest-web2 --network nettest-net --network-alias webalias nginx:alpine
docker run --rm --network nettest-net alpine wget -qO- http://webalias       # works
```

5.3 DNS options for docker run

Flag	Effect
`--dns 8.8.8.8`	Use this DNS server for external names
`--dns-search example.com`	Search domain for short names
`--dns-option ndots:2`	Extra resolver option
`--hostname web01`	Host name inside the container
`--add-host db.local:10.0.0.5`	Adds a line to the container's `/etc/hosts`

5.4 Reaching the host from a container

`localhost` inside a container is the container itself, not your laptop. To call a service running on the host (for example an app started with `mvn spring-boot:run`):

- **Docker Desktop** (Windows/macOS): use `host.docker.internal` - it works out of the box.
- **Linux**: add `--add-host=host.docker.internal:host-gateway` to `docker run`.

Container -> app published on the host

```
docker run --rm alpine wget -qO- http://host.docker.internal:9090/name
Welcome to TrishaAcademy
```

Common mistake

Configuring `jdbc:mysql://localhost:3306` in a container and expecting it to reach a database on your laptop or in another container. Use the container name (same network) or `host.docker.internal` (service on the host).

DOCKER NETWORKING · PART 06

Managing networks

Create, connect, inspect and remove networks with the Docker CLI.

6.1 The docker network commands

Command	What it does
`docker network create NAME`	Create a user-defined bridge network
`docker network ls`	List networks
`docker network inspect NAME`	Subnet, gateway, options and attached containers
`docker network connect NAME CONTAINER`	Attach a running container to another network
`docker network disconnect NAME CONTAINER`	Detach it again
`docker network rm NAME`	Delete a network (no container may be attached)
`docker network prune`	Delete all unused networks

6.2 Subnet, gateway, static IP and internal networks

Custom subnet and a fixed IP

```
docker network create --subnet 172.30.0.0/24 --gateway 172.30.0.1 nettest-sub
docker run --rm --network nettest-sub --ip 172.30.0.50 alpine ip -o -4 addr show eth0
2: eth0    inet 172.30.0.50/24 brd 172.30.0.255 scope global eth0
```

Internal network

```
docker network create --internal nettest-int
docker run --rm --network nettest-int alpine wget -qO- -T 4 http://example.com
wget: bad address 'example.com'          # no way out to the internet
```

Option	Use
`--driver bridge`	Driver (bridge is the default)
`--subnet`, `--gateway`, `--ip-range`	Choose the address range - avoid clashes with office/VPN networks
`--ip` **(on** `docker run`**)**	Fixed IP - only on a user-defined network with `--subnet`
`--internal`	No outbound traffic - good for databases
`--ipv6`	Enable IPv6 on the network
`-o com.docker.network.bridge.name=br-trisha`	Name of the bridge interface on the Linux host

6.3 One container, several networks

A container can be on several networks at the same time - the classic pattern is a front-end network for the web app and a back-end network shared only with the database.

Connect and disconnect a running container

```
docker network connect nettest-net nettest-web1
docker inspect -f "{{range $k,$v := .NetworkSettings.Networks}}{{$k}}={{$v.IPAddress}} {{end}}" nettest-w
eb1
bridge=172.17.0.2 nettest-net=172.18.0.3
docker network disconnect nettest-net nettest-web1
```

Did you know?
Since Docker Engine 29 the old shortcut `{{.NetworkSettings.IPAddress}}` is gone - read the IP per network as shown above, or look at `docker inspect` output under `Networks`.

6.4 Clean up

PowerShell

```
docker rm -f nettest-web1 nettest-web2 nettest-web3 nettest-web4
docker network rm nettest-net nettest-int nettest-sub
docker network prune          # removes every network no container uses
```

DOCKER NETWORKING · PART 07

host, none and container: networking

Turn isolation off, switch networking off, or share one container's network.

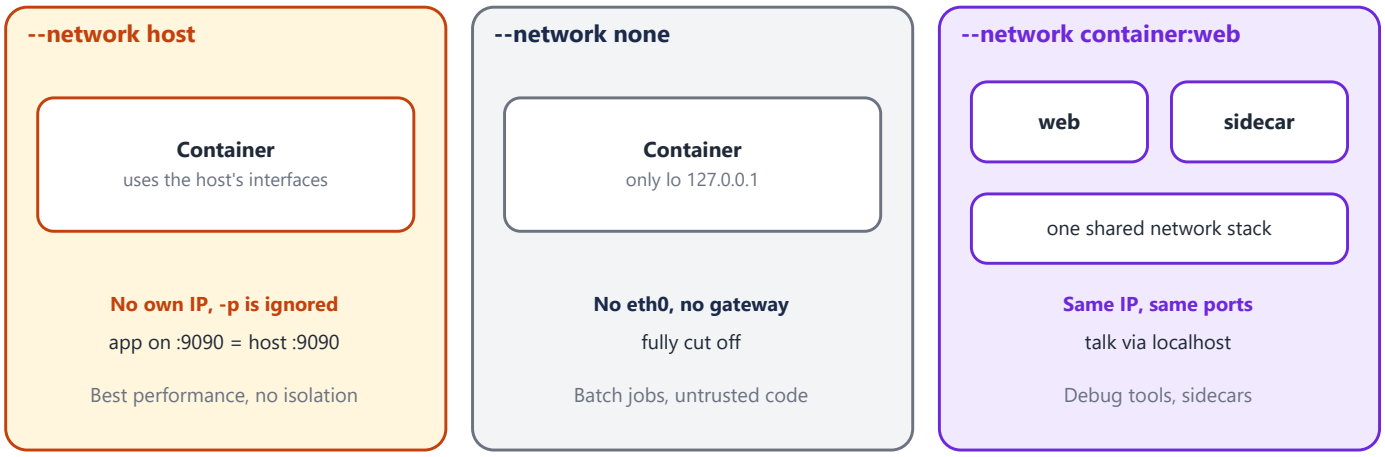


Figure 7.1 - Three special network modes

7.1 host

From the official Docker documentation
If you use the host network mode for a container, that container's network stack isn't isolated from the Docker host (the container shares the host's networking namespace), and the container doesn't get its own IP-address allocated.

Linux host

```
docker run -d --name web-host --network host nginx:alpine
# nginx now listens on port 80 of the host itself - no -p needed (and -p is ignored)
```

- Fastest option: no NAT and no port forwarding.
- Port clashes: two containers cannot both listen on port 80.
- On Docker Desktop host networking is an opt-in setting in newer versions, and it means the Docker Desktop VM's network - use `-p` on Windows/macOS instead.

7.2 none

Only the loopback interface

```
docker run --rm --network none alpine ip -o addr
1: lo    inet 127.0.0.1/8 scope host lo
```

7.3 container:<name>

The new container joins the network namespace of an existing one: same IP, same ports, and they talk over `localhost`. Useful for debugging a container that has no tools inside.

Share the network of nettest-web2

```
docker run --rm --network container:nettest-web2 alpine wget -qO- http://localhost
<!DOCTYPE html> ... Welcome to nginx! ...
```

Try it yourself
Start `nginx:alpine` without any port, then use `--network container:` with `alpine` to fetch its page over `localhost`.

DOCKER NETWORKING · PART 08

macvlan and ipvlan

Put containers directly on the physical network (Linux hosts).

8.1 macvlan

From the official Docker documentation

Some applications, especially legacy applications or applications which monitor network traffic, expect to be directly connected to the physical network. In this type of situation, you can use the macvlan network driver to assign a MAC address to each container's virtual network interface, making it appear to be a physical network interface directly connected to the physical network.

Linux host, eth0 on the 192.168.1.0/24 LAN

```
docker network create -d macvlan `
  --subnet=192.168.1.0/24 --gateway=192.168.1.1 `
  -o parent=eth0 pub_net
docker run -d --name legacy-app --network pub_net --ip 192.168.1.50 nginx:alpine
```

- The container gets an address on your real LAN - other machines reach it without `-p`.
- The host itself usually **cannot** talk to its macvlan containers directly.
- Many switches and cloud networks (AWS, Azure) block unknown MAC addresses - check first.
- Pick an IP range that your DHCP server does not hand out.

8.2 ipvlan

Like macvlan, but every container shares the host's MAC address. Use it when the network limits the number of MAC addresses per port.

Mode	How it works
`ipvlan` **L2 (default)**	Containers are on the same subnet as the host, like macvlan
`ipvlan` **L3**	The host routes between container subnets - no broadcast, needs routes on the router

ipvlan L2

```
docker network create -d ipvlan --subnet=192.168.1.0/24 --gateway=192.168.1.1 `
  -o parent=eth0 -o ipvlan_mode=l2 ipvlan_net
```

Common mistake

Trying macvlan or ipvlan on Docker Desktop for Windows/macOS. They need a Linux host with a real network interface. On a laptop, stay with bridge networks and `-p`.

DOCKER NETWORKING · PART 09

Hands-on lab: network the devops-java-demo app

Run the Student API on its own network, call it by name from another container, and control who can reach it.

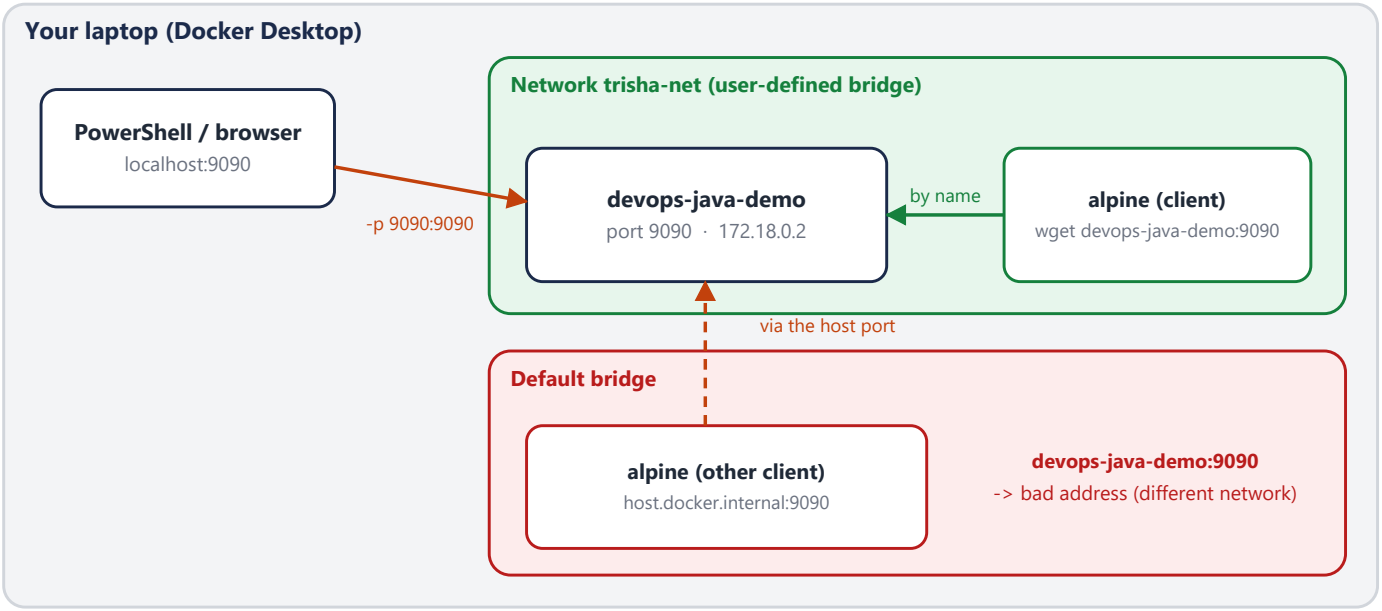


Figure 9.1 - The lab setup (every arrow was tested)

9.1 Build the image and create the network

PowerShell

```
cd D:\TRISHA-ACADEMY\DevOPs\devops-java-demo
docker build -t devops-java-demo:1.0.0 .
docker network create trisha-net
```

9.2 Run the app on trisha-net and publish port 9090

PowerShell

```
docker run -d --name devops-java-demo --network trisha-net -p 9090:9090 `
  -v "${PWD}\data:/app/data" devops-java-demo:1.0.0
docker logs -f devops-java-demo        # wait for "Started SpringLabApplication", then Ctrl + C
```

9.3 Call it from three places

Expected output

```
# 1. From the host (published port)
Invoke-RestMethod http://localhost:9090/name
Welcome to TrishaAcademy

# 2. From another container on trisha-net (by name, container port)
docker run --rm --network trisha-net alpine wget -qO- http://devops-java-demo:9090/name
Welcome to TrishaAcademy

# 3. From a container on the default bridge
docker run --rm alpine wget -qO- -T 3 http://devops-java-demo:9090/name
wget: bad address 'devops-java-demo:9090'
docker run --rm alpine wget -qO- http://host.docker.internal:9090/name
Welcome to TrishaAcademy
```

9.4 Keep the app private

Run it again **without** `-p`: other containers on `trisha-net` can still use it, but the host and the outside world cannot.

Expected output

```
docker rm -f devops-java-demo
docker run -d --name devops-java-demo --network trisha-net `
  -v "${PWD}\data:/app/data" devops-java-demo:1.0.0

docker run --rm --network trisha-net alpine wget -qO- http://devops-java-demo:9090/name
Welcome to TrishaAcademy
Invoke-RestMethod http://localhost:9090/name -TimeoutSec 3
Invoke-RestMethod : The operation has timed out.
```

9.5 Results

#	From	Address	Result
1	Host (with `-p 9090:9090`)	`localhost:9090`	Works
2	Container on `trisha-net`	`devops-java-demo:9090`	Works - embedded DNS
3	Container on default bridge	`devops-java-demo:9090`	**Fails** - different network
4	Container on default bridge	`host.docker.internal:9090`	Works - through the host port
5	Container on `trisha-net` (no `-p`)	`devops-java-demo:9090`	Works
6	Host (no `-p`)	`localhost:9090`	**Fails** - port not published

9.6 Variations to try

- Publish only to your laptop: `-p 127.0.0.1:9090:9090`.
- Run a second copy on another host port: `--name devops-java-demo-2 -p 9091:9090` - the container port stays 9090.
- Give the app an alias: `--network-alias student-api`, then call `http://student-api:9090/name`.
- Make the network internal (`docker network create --internal trisha-private`) - the app keeps working for other containers but cannot reach the internet.

9.7 Clean up

PowerShell

```
docker rm -f devops-java-demo
docker network rm trisha-net
```

DOCKER NETWORKING · PART 10

Best practices, troubleshooting and cheat sheet

Avoid the classic mistakes.

10.1 Best practices

- Create a **user-defined bridge** per application; never rely on the default bridge.
- Call other containers by **name** or alias - never hard-code container IP addresses.
- Publish only the ports users need; keep databases unpublished on an internal network.
- Publish to `127.0.0.1` on shared or public machines unless the port must be public.
- Choose subnets that do not clash with your office, VPN or cloud networks.
- Use `host.docker.internal` (Docker Desktop) or `host-gateway` (Linux) to reach host services.
- Remove unused networks with `docker network prune`.

10.2 Troubleshooting

Problem	Cause and fix
`wget: bad address 'name'`	Containers are on different networks, or on the default bridge - put both on the same user-defined network
Browser cannot open `localhost:9090`	No `-p`, wrong port order (`HOST:CONTAINER`), or the app is not started yet - check `docker port` and `docker logs`
`port is already allocated`	Another container or program uses the host port - pick another host port, e.g. `-p 9091:9090`
App unreachable although published	The app listens on `127.0.0.1` inside the container - it must listen on `0.0.0.0` (Spring Boot does by default)
`network ... has active endpoints`	Containers still attached - `docker network inspect`, then remove or disconnect them
`Pool overlaps with other one`	Subnet clash - choose another `--subnet`
`localhost` **in a container does not reach the host**	Use `host.docker.internal`
Template error on `.NetworkSettings.IPAddress`	Docker 29+ - use `.NetworkSettings.Networks.<net>.IPAddress`

10.3 Cheat sheet

Goal	Command
Create / list / inspect	`docker network create n` · `docker network ls` · `docker network inspect n`
Run on a network	`docker run --network n --name app image`
Alias	`--network-alias api`
Publish	`-p 9090:9090` · `-p 127.0.0.1:9090:9090` · `-P`
Which ports are published	`docker port c`
Attach / detach	`docker network connect n c` · `docker network disconnect n c`
IP of a container	`docker inspect -f "{{.NetworkSettings.Networks.n.IPAddress}}" c`
Test from inside	`docker run --rm --network n alpine wget -qO- http://app:9090`
Special modes	`--network host` · `--network none` · `--network container:c`
Remove	`docker network rm n` · `docker network prune`

DOCKER NETWORKING · PART 11

Top 20 interview questions

Answer aloud before reading the model answer.

Interviewers look for a clear definition, one real example and one trade-off.

Q1. What is the default network driver?

bridge. Containers started without --network join the default bridge network called bridge (docker0, 172.17.0.0/16 by default).

Q2. Name the built-in network drivers.

bridge, host, none, macvlan and ipvlan, plus third-party network plugins.

Q3. Default bridge vs user-defined bridge?

A user-defined bridge gives automatic DNS by container name, better isolation, attach/detach while running, and per-network configuration. The default bridge only allows IP addresses (or legacy --link).

Q4. How do two containers talk to each other?

Put them on the same user-defined network and use the container name (or an alias) with the container port, e.g. http://devops-java-demo:9090.

Q5. What does -p 8080:80 mean?

Traffic to host port 8080 is forwarded to port 80 inside the container. Left side = host, right side = container.

Q6. Does EXPOSE publish a port?

No. EXPOSE only documents the port (and is used by -P). You must publish with -p or -P.

Q7. Why is publishing a port considered insecure by default?

-p 8080:80 binds to all host interfaces, so the port is reachable from other machines. Use -p 127.0.0.1:8080:80 to allow only the host.

Q8. What is 127.0.0.11?

Docker's embedded DNS server in containers on user-defined networks. It resolves container names and aliases and forwards other names to the host's DNS servers.

Q9. What is a network alias?

An extra DNS name for a container on a network (--network-alias). Several containers can share one alias.

Q10. What does --network host do?

The container shares the host's network stack: no own IP, no NAT, -p is ignored, and the app's ports are the host's ports. Fast but not isolated.

Q11. When would you use --network none?

For containers that must have no network at all, such as batch jobs or untrusted code. Only loopback exists.

Q12. What is --network container:<name>?

The new container joins another container's network namespace: same IP and ports, communication over localhost. Useful for debugging and sidecars.

Q13. How does a container reach a service on the host?

Use host.docker.internal on Docker Desktop; on Linux add --add-host=host.docker.internal:host-gateway. localhost inside a container is the container itself.

Q14. Can a container be on more than one network?

Yes. Use docker network connect / disconnect, e.g. a web app on a front-end and a back-end network.

Q15. What is an internal network?

A network created with --internal: containers on it can talk to each other but have no outbound internet access.

Q16. How do you give a container a fixed IP?

Create a user-defined network with --subnet, then docker run --network <net> --ip <address>.

Q17. macvlan vs ipvlan?

Both put containers directly on the physical network. macvlan gives each container its own MAC address; ipvlan shares the host's MAC and supports L2 and L3 modes.

Q18. How do you find a container's IP address?

docker inspect -f "{{.NetworkSettings.Networks.<net>.IPAddress}}" <container>, or look under Networks in docker inspect.

Q19. Why should you not hard-code container IPs?

IPs change when containers are recreated. Names and aliases are resolved by Docker's DNS every time.

Q20. Your app in a container is published but not reachable - what do you check?

docker ps / docker port for the mapping, docker logs for start-up, that the app listens on 0.0.0.0 and the right port, the host port is free, and the firewall.

Reference

- Docker docs - Networking overview: https://docs.docker.com/engine/network/
- Docker docs - Bridge network driver: https://docs.docker.com/engine/network/drivers/bridge/
- Docker docs - Port publishing: https://docs.docker.com/engine/network/port-publishing/
- Docker docs - host, none, macvlan, ipvlan: https://docs.docker.com/engine/network/drivers/
- Companion notes: Docker-Volumes-Guide.pdf